Three failed logins. One successful sign-in. What happens next?

A hands-on journey from a suspicious pattern to tested response controls.

THE CHALLENGE

Was someone struggling with a password, or had an account just been accessed without permission?

THE WORK

I used KQL to investigate synthetic events, Jira to document the case, and Python to test alerting, containment and recovery.

THE RESULT

One replay alert. A rejected old session. A blocked disabled account. A verified recovery path.

Read the evidence. Follow the decisions. Learn what each result can - and cannot - tell us.

Scope: fictional users and events in a personal lab. Response controls were exercised in an isolated training application; no production accounts were changed.

Give the investigation somewhere to start.

Before investigating a login, I needed a place to store and query the records.

Azure Data Explorer held the practice data. **Kusto** is the underlying technology; **KQL** is the language used to ask questions of the data.

I set up a free cluster in Europe, used the SOCTraining database, defined the LoginLogs table and ingested synthetic authentication events. ApplicationLogs supported the later application-activity review.

```
1  .ingest inline into table LoginLogs <|
2  2026-09-23T09:00:00Z,alex,192.0.2.10,Failed,DeliveryPortal
```

ExtentId	ItemLoaded	Duration	HasErrors	OperationId
c74d0419-1df7-47ef-9	inproc:c9cb7fb0-462l	00:00:00.2849599	false	249f72ee-badf-4000-

Evidence 1 | Ingestion returned HasErrors = false. The input is deliberately fictional; successful ingestion confirms loading, not authenticity.

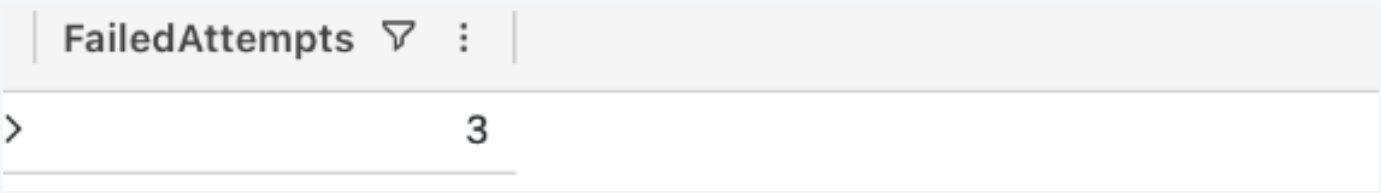
WHAT A LOG GIVES YOU

A timestamp, a user, an IP address, an application and an outcome: small facts that become useful when examined together.

Ask a small question. Get a clear answer.

How many failed login attempts are recorded for Alex?

```
1  LoginLogs
2  | where UserName == "alex"
3  | where Result == "Failed"
4  | summarize FailedAttempts = count()
```



Evidence 2 | The query filters Alex's failed events and returns FailedAttempts = 3.

where keeps records that match a condition.
summarize combines them into a result.
count() tells us how many matching records exist.

THREE FAILURES ARE A CLUE

The screenshot proves the count in the selected data. It does not establish an attack, a time window or a successful login afterward.

I also practised filtering by account and IP, sorting by time, counting outcomes and comparing authentication with application activity.

Build the story.
Keep the gaps visible.

The fictional case combined failed logins, successful access and a blocked export attempt.

- **09:00-09:02** — **Three failed logins**
  Suspicious, but still compatible with mistakes.

- **09:03** — **Successful login**
  A reason to investigate who gained access.

- **09:04** — **Order-1042 viewed**
  The supplied application record shows a view.

- **09:05** — **Customer export denied**
  That attempt was blocked; scope is still limited.

- **09:06** — **Logout recorded**
  Not proof that every token became invalid.

Scenario event times: 23 September 2026, UTC. These are not screenshot capture times.

THE IMPORTANT LIMIT

The original tables had no shared session ID. Matching user and time supports a hypothesis; it does not prove the same session performed every action.

Make the reasoning easy to follow.

Jira turned observations, questions and next actions into a case record.

SOC-2 tracked the main investigation. SOC-3 tracked how access occurred and was linked to the main case. I documented findings, uncertainties and follow-up work rather than treating each alert as a disconnected task.

FROM THE LAB CASE NOTE

How the password was obtained remains undetermined; guessing, phishing, and reuse are unconfirmed possibilities.

Transcribed excerpt from the uploaded Jira investigation screenshot. The original screenshot is omitted because it includes unrelated personal browser content.

A USEFUL CASE NOTE ANSWERS

What do we know? What supports it? What remains uncertain? Who owns the next action?

The MFA exception and session details in this comment are scenario evidence. They were not retrieved from a live identity provider. Final main-case closure is not shown by the selected screenshots.

Turn a pattern into an alert.

The Python exercise added sequence, time and duplicate handling to the initial counting work.

Rule: at least three distinct failures strictly before a success, within five minutes, for the same user, IP and application. A repeated delivery of the same success must not create another alert.

```
AUTOMATIC ALERT: alex — 3 failures before success
PASS: one alert, repeated delivery suppressed, worker stopped.
{
  "alert_id": "rule1:s1",
  "user": "alex",
  "ip": "192.0.2.10",
  "application": "DeliveryPortal",
  "failures": 3,
  "success_time": "2026-09-23T09:03:00+00:00",
  "evidence_ids": [
    "f1",
    "f2",
    "f3",
    "s1"
  ],
  "disposition": "Needs analyst review"
}
```

Evidence 3 | The replay worker emitted one alert, suppressed repeated delivery and stopped.

TEST THE EDGES, TOO

The notebook includes ten checks: time boundaries, event order, duplicates, different IPs and a legitimate pattern that still triggers the rule.

A short replay is not continuous SIEM monitoring. The fixture checks demonstrate rule behaviour; they do not measure production detection quality.

A success message is not enough.

After changing a control, try the access that should now fail.

In the isolated Python application, I cleared active sessions and disabled the training account. Then the notebook retried the old session and attempted a fresh login.

```
sessions.clear()
record("admin-revoke-sessions","completed")
account["enabled"]=False
record("admin-disable-account","completed")
old_access=request("/order",token=old_token)[0]
fresh_login=request("/login",{"user":"alex","password":OLD_PASSWORD})[0]
assert old_access==401 and fresh_login==403
print("CONTAINED: old session",old_access,"; fresh login",fresh_login)
print("Verified by rejected requests, not merely by an admin success message.")
```

```
CONTAINED: old session 401 ; fresh login 403
Verified by rejected requests, not merely by an admin success message.
```

Evidence 4 | The tested old session returned 401; the fresh login returned 403.

READ THE RESULTS

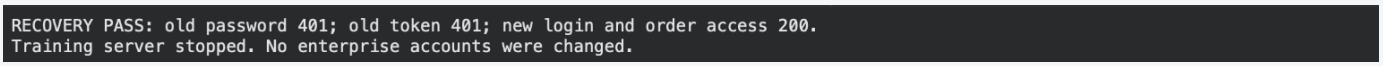
200 = request succeeded.
401 = authentication rejected.
403 = access forbidden by the application.

This verified containment for the tested paths in the training app. It does not establish organisation-wide revocation or real identity-platform administration.

Restore good access. Keep old access out.

Recovery needs both positive and negative checks.

I reset the disposable training password and re-enabled the account. The exercise checked that the old password and old session remained unusable while new credentials could access the protected order.



Evidence 5 | Recovery output: old password 401; old token 401; new login and order access 200.

OLD PASSWORD / OLD SESSION

Both rejected. Restoring access did not reactivate the tested old credentials or token.

NEW PASSWORD / NEW SESSION

Accepted. The intended user journey worked again in the training application.

The server was stopped after the exercise. The app has no real MFA, enterprise identity integration or production session-management controls.

An alert starts a question.

The strongest outcome is a decision you can explain with evidence.

Threshold is the trigger. Three failures in five minutes is a condition to investigate.

Fidelity is usefulness. A detection needs enough context to separate meaningful threats from ordinary activity. A higher threshold may reduce noise while missing attacks.

The Taylor exercise made this concrete. A legitimate person can match the same rule. A verified callback, device record, MFA event and corresponding application activity would help assess legitimacy. In that scenario, those details were assumptions - no real verification occurred.

MY RESPONSE PLAYBOOK

Validate the alert. Preserve evidence. Scope activity. Verify context. Contain when justified. Test the controls. Recover. Document and improve.

Skills practised: KQL investigation, timeline analysis, case ownership, detection testing, event automation, response verification and clear incident communication.

Reproduce the lab with the companion notebook and review the evidence register for source screenshots. This project demonstrates a complete training workflow with its limits stated clearly.